

```
// convert time from "HH:MM" format to total minutes
```

```
function timeToMinutes(time):
```

```
    hour = extract hour part and convert to integer
```

```
    minute = extract minute part and convert to integer
```

```
    return hour * 60 + minute // return total minutes
```

```
// convert total minutes to "HH:MM" format
```

```
function minutesToTime(minutes):
```

```
    hour = minutes / 60 // calculate hours
```

```
    minute = minutes % 60 // calculate remaining minutes
```

```
    format as "HH:MM"
```

```
    return formatted time
```

```
// find overlapping periods between two lists of time intervals
```

```
function computeIntersection(times1, times2):
```

```
    result = empty list
```

```
    for each t1 in times1:
```

```
        for each t2 in times2:
```

```
            if t1 and t2 overlap:
```

```
                start = max(t1.start, t2.start) // get overlap start
```

```
                end = min(t1.end, t2.end) // get overlap end
```

```
                add (start, end) to result
```

```
    return result // return list of overlapping periods
```

```
// merge overlapping or adjacent time intervals
```

```
function computeUnion(times):
```

```
if times is empty, return empty list

sort times by start time

result = empty list

current = first interval

for each time in times:

    if time overlaps with current:

        update current to merge with time

    else:

        add current to result

        current = time        // move to the next interval

add current to result    // add the last interval

return result // return merged intervals
```

// subtract one set of intervals from another

function subtractTimes(times, subtract):

```
result = empty list

for each time in times:

    start = time.start

    for each sub in subtract:

        if sub overlaps with time:

            if start < sub.start:

                add (start, sub.start) to result

            update start = sub.end

    if start < time.end:

        add (start, time.end) to result    // add remaining part

return result    // return the result after subtraction
```

```

// parse input string into a list of time intervals

function parseTimes(line):
    remove '[' ']' and '\' from line
    split line by commas into times
    schedule = empty list
    for i from 0 to length of times - 1 step 2:
        add (times[i], times[i + 1]) to schedule
    return schedule // return parsed schedule

// main function to found suitable meeting times

main:
    print "person1_Schedule = "
    input busyLine1
    print "person1_DailyAct = "
    input freeLine1

    print "person2_Schedule = "
    input busyLine2
    print "person2_DailyAct = "
    input freeLine2

    busy1 = parseTimes(busyLine1)
    free1 = parseTimes(freeLine1)
    busy2 = parseTimes(busyLine2)
    free2 = parseTimes(freeLine2)

```

busyPeriods = empty list

for each p in busy1:

 add (timeToMinutes(p.first), timeToMinutes(p.second)) to busyPeriods

for each p in busy2:

 add (timeToMinutes(p.first), timeToMinutes(p.second)) to busyPeriods

busyPeriods = computeUnion(busyPeriods)

freePeriods = empty list

freePeriods2 = empty list

for each p in free1:

 add (timeToMinutes(p.first), timeToMinutes(p.second)) to freePeriods

for each p in free2:

 add (timeToMinutes(p.first), timeToMinutes(p.second)) to freePeriods2

freePeriods = computeIntersection(freePeriods, freePeriods2)

availablePeriods = subtractTimes(freePeriods, busyPeriods)

print "duration_of_meeting = "

input duration

outputPeriods = empty list

for each period in availablePeriods:

 if period duration >= duration:

 add "[" + minutesToTime(period.first) + ";" + minutesToTime(period.second) + "]" to
outputPeriods

```

if outputPeriods.size == 1:
    print outputPeriods[0]
else if outputPeriods.size > 1:
    print "["
    for i from 0 to outputPeriods.size - 1:
        print outputPeriods[i]
        if i < outputPeriods.size - 1:
            print ","
    print "]"

```

Time Complexity Analysis

Input:

getline: Each line is read from input. This operation takes **$O(n)$** , n is the size of the input.

parseTimes Function: It iterates through the input line, splitting it by commas and removing unnecessary characters. If there are m time intervals, the complexity is **$O(m)$** .

Converting Time Strings to Minutes:

timeToMinutes function: This operation is **$O(1)$** for each conversion since the string operations are constant time.

Union of Busy Periods:

computeUnion function:

Sorting the intervals **takes $O(n \log n)$** time, where n is the number of busy intervals.

Merging overlapping intervals takes **$O(n)$** time.

Total: **$O(n \log n)$** .

Intersection of Free Periods:

computeIntersection function: This involves comparing every interval from both input lists.

If the first list **has m** intervals and the second list **has k** intervals, the complexity **is $O(m*k)$** .

Subtracting Busy Periods from Free Periods:

subtractTimes function: If there are m free intervals and n busy intervals, the complexity **is $O(m*n)$** .

Generating Output:

Iterating through available periods and checking the duration takes $O(m)$, where m is the number of available periods.

Overall Time Complexity:

Dominant Factors: Sorting in the union operation **$O(n \log n)$** and the intersections/subtraction operations **$O(m*n)$** .

Overall: $O(n \log n + m*n)$.

Space Complexity Analysis

Input Storage: If there are m intervals for person 1 and k for person 2, the space complexity **is $O(m+k)$** for storing the schedules.

Converted Time Intervals:

All time intervals are stored as pairs of integers, **requiring $O(m+n)$** space.

Union and Intersection Results:

The union, intersection, and subtraction operations create new lists of intervals, each **$O(m+n)$** space in the worst case.

Output Storage:

The final list of available periods is stored in outputPeriods, taking $O(m)$ space.

Overall Space Complexity: $O(m+n)$